Regular Expressions

```
are special text strings to find patterns in data
```

Import Re Module (RegEx)

```
import re
```

Methods in Re Module

```
match = re.match(substring, string, re.I)
# search object at index 0, if not found, return None
#OR
match = re.match("word | Word", string)
#OR
match = re.match("[Ww]ord", string)
    # re.I to ignore case, case insensitive
span = match.span()
# span return tuple (0, end) index of the match

match = re.search(substring, string, re.I)
span = match.span()
# search anywhere in the string, if not found, return None

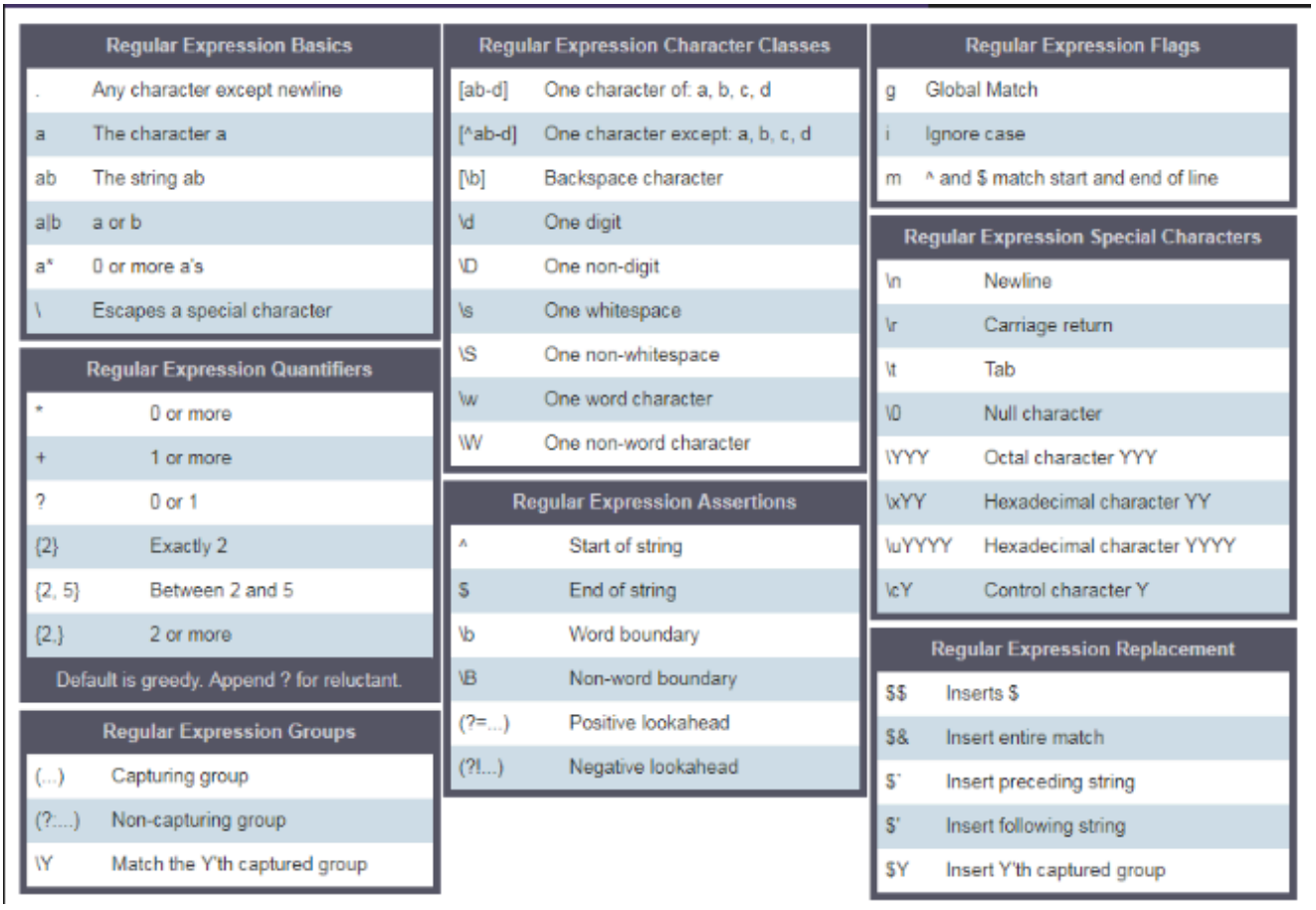
re.findall(substring, string, re.I)
# return a list of all matches

re.split(separator, text)
# split strings at match point, return a list

re.sub(object_to_sub, sub_to, string, re.I)
# replace one of many matches in a string
```

RegEx Variables, Patterns

> **Important**
>
> look at obsidian for patterns



Regular Expression Basics

.	Any character except newline
a	The character a
ab	The string ab
a|b	a or b
a*	0 or more a's
\	Escapes a special character

Regular Expression Quantifiers

*	0 or more
+	1 or more
?	0 or 1
{2}	Exactly 2
{2, 5}	Between 2 and 5
{2,}	2 or more

Default is greedy. Append ? for reluctant.

Regular Expression Groups

(...)	Capturing group
(?:...)	Non-capturing group
\Y	Match the Y'th captured group

Regular Expression Character Classes

[ab-d]	One character of: a, b, c, d
[^ab-d]	One character except: a, b, c, d
[\b]	Backspace character
\d	One digit
\D	One non-digit
\s	One whitespace
\S	One non-whitespace
\w	One word character
\W	One non-word character

Regular Expression Assertions

^	Start of string
$	End of string
\b	Word boundary
\B	Non-word boundary
(?=...)	Positive lookahead
(?!...)	Negative lookahead

Regular Expression Flags

g	Global Match
i	Ignore case
m	^ and $ match start and end of line

Regular Expression Special Characters

\n	Newline
\r	Carriage return
\t	Tab
\0	Null character
\YYY	Octal character YYY
\xYY	Hexadecimal character YY
\uYYYY	Hexadecimal character YYYY
\cY	Control character Y

Regular Expression Replacement

$$	Inserts $
$&	Insert entire match
$`	Insert preceding string
$'	Insert following string
$Y	Insert Y'th captured group

Quantifier for RegEx

```
regex_pattern = r"\d{4}"
# repeat four times
regex_pattern = r"\d{1, 4}"
# repeat one, four, one fout
```

Cart ^

Starts With

```
r"^This"
# starts with This
```

Negation

```
[^..]
# not this
```

Combination of f"" and r"

```
rf"\b{re.escape(word)}\b"
```